Homework 4: Modern Statistics for Engineers

Completed by Brandon Auyeung for Dr. Peyman Abbaszadeh

Due Date: 9th of March, 2026

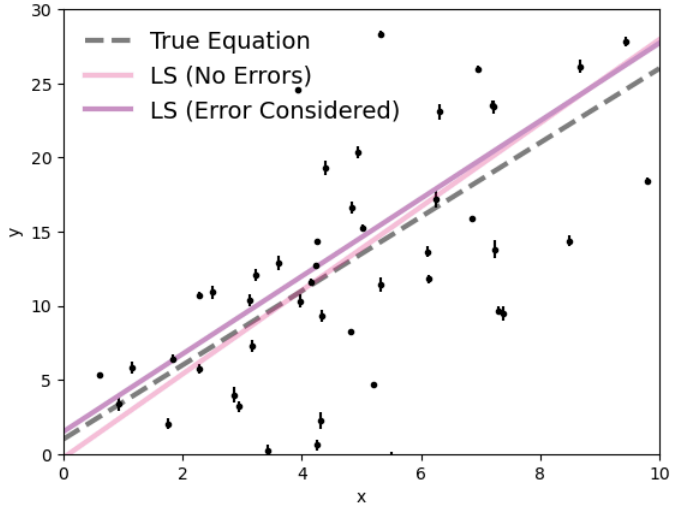
Problem 1

In section "Generative Probabilistic Model", we used w = np.linalg.solve(ATA, np.dot(A.T, y / yerr**2)" to find the model parameters and fit a line to data. Use what we learned in Chapter 2, Least Square Fitting, to find the model parameters and fit a line to data. Provide your Python script and visualize the results like what we did in Chapter 2, page 8. (2 points)

In [45]:
```
np.random.seed(123)

# Choose the "true" parameters.
m_true = 2.5
b_true = 1
f_true = 0.5

# Generate some synthetic data from the model.
N = 50
x = np.sort(10 * np.random.rand(N))
yerr = 0.1 + 0.5 * np.random.rand(N)

y = m_true * x + b_true
y += np.abs(f_true * y) * np.random.randn(N)
y += yerr * np.random.randn(N)
```

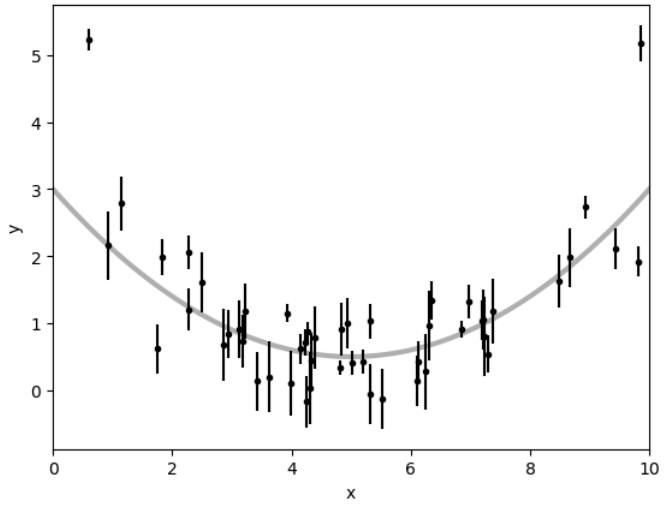
In [54]:
```
A = np.vander(x, 2)
C = np.diag(yerr * yerr)
ATA = np.dot(A.T, A / (yerr**2)[:, None])
cov = np.linalg.inv(ATA)
w = np.linalg.solve(ATA, np.dot(A.T, y / yerr**2))
print("Least-squares estimates with error consideration:")
print("m = {0:.3f} ± {1:.3f}".format(w[0], np.sqrt(cov[0, 0])))
print("b = {0:.3f} ± {1:.3f}".format(w[1], np.sqrt(cov[1, 1])))
print()

x_mean = np.mean(x)
y_mean = np.mean(y)
xy_mean = np.mean(x * y)
x_squared_mean = np.mean(x**2)
a_est = (xy_mean - x_mean * y_mean) / (x_squared_mean - x_mean**2)
b_est = y_mean - a_est * x_mean
print("Least squares estimates (not considering errors):")
print(f"m: {a_est}")
print(f"b: {b_est}")


plt.errorbar(x, y, yerr=yerr, fmt=".k", capsize=0)
plt.plot(x0, m_true * x0 + b_true, color ="#000000",linestyle= "--", lw=3, alpha = 0.5, label="True Equation")
plt.plot(x0, a_est * x0 + b_est, "#e7298a", alpha=0.3, lw=3, label= "LS (No Errors)")
plt.plot(x0, np.dot(np.vander(x0, 2), w), "#c994c7", lw = 3, label="LS (Error Considered)")
plt.legend(fontsize=14, frameon=False)
plt.xlim(0, 10)
plt.ylim(0, 30)
plt.xlabel("x")
plt.ylabel("y");
```

```
Least-squares estimates with error consideration:
m = 2.621 ± 0.016
b = 1.505 ± 0.091

Least squares estimates (not considering errors):
m: 2.82116217391037
b: -0.2516851009452026
```



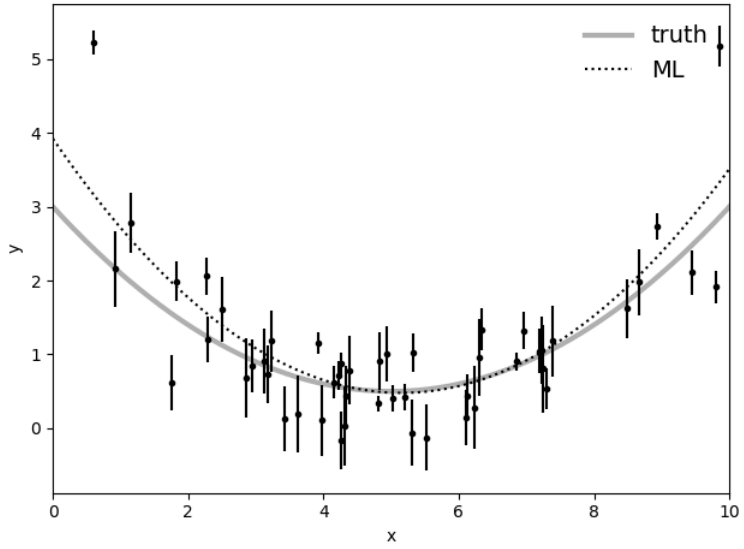
Problem 2

In [32]:
```
np.random.seed(123)

# Choose the "true" parameters for a quadratic model.
a_true = 0.1
b_true = -1.0
c_true = 3.0
f_true = 0.5

# Generate some synthetic data from the quadratic model.
N = 50

x = np.sort(10 * np.random.rand(N))
yerr = 0.1 + 0.5 * np.random.rand(N)
y = a_true * x**2 + b_true * x + c_true
y += np.abs(f_true * y) * np.random.randn(N)
y += yerr * np.random.randn(N)

# Create a scatter plot with error bars
plt.errorbar(x, y, yerr=yerr, fmt=".k", capsize=0)

# Generate the quadratic curve based on the "true" parameters
x0 = np.linspace(0, 10, 500)
y0 = a_true * x0**2 + b_true * x0 + c_true

# Plot the quadratic curve
plt.plot(x0, y0, "k", alpha=0.3, lw=3)

plt.xlim(0, 10)
plt.xlabel("x")
plt.ylabel("y")

# Show the plot
plt.show()
```



In [33]:
```
def log_likelihood(theta, x, y, yerr):
    a,b,c, log_f = theta
    model = a * x**2 + b *x + c
    sigma2 = yerr**2 + model**2 * np.exp(2 * log_f)
    return -0.5 * np.sum((y - model) ** 2 / sigma2 + np.log(sigma2))

np.random.seed(123)
nll = lambda *args: -log_likelihood(*args)
initial = np.array([a_true, b_true, c_true, np.log(f_true)]) + 0.1 * np.random.randn(4)
soln = minimize(nll, initial, args=(x, y, yerr))
a_ml, b_ml, c_ml, log_f_ml = soln.x

print("Maximum likelihood estimates:")
print("a = {0:.3f}".format(a_ml))
print("b = {0:.3f}".format(b_ml))
print("c = {0:.3f}".format(c_ml))
print("f = {0:.3f}".format(np.exp(log_f_ml)))

plt.errorbar(x, y, yerr=yerr, fmt=".k", capsize=0)
plt.plot(x0, a_true * x0**2 + b_true * x0 + c_true, "k", alpha=0.3, lw=3, label="truth")
plt.plot(x0, np.dot(np.vander(x0, 3), [a_ml, b_ml, c_ml]), ":k", label="ML")
plt.legend(fontsize=14, frameon=False)
plt.xlim(0, 10)
plt.xlabel("x")
plt.ylabel("y")
plt.tight_layout()
plt.show()
```

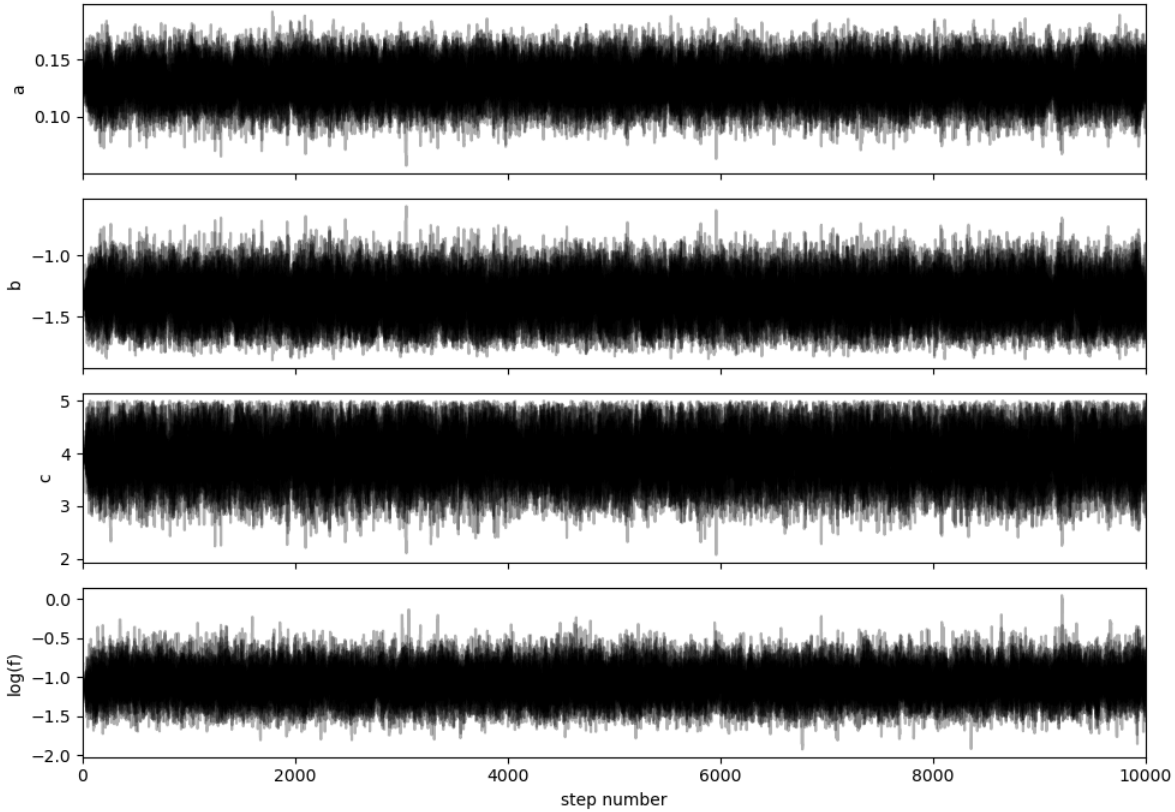
```
Maximum likelihood estimates:
a = 0.129
b = -1.334
c = 3.921
f = 0.323
```



In [34]:
```
def log_prior(theta):
    a, b, c, log_f = theta
    if 0 < a < 3 and -6 < b < 6 \
        and 2 < c < 5:
        return 0.0
    return -np.inf

def log_probability(theta, x, y, yerr):
    lp = log_prior(theta)
    if not np.isfinite(lp):
        return -np.inf
    return lp + log_likelihood(theta, x, y, yerr)

import emcee

np.random.seed(123)

pos = soln.x + 0.01 * np.random.randn(32, 4)
print(pos.shape)
nwalkers, ndim = pos.shape

sampler = emcee.EnsembleSampler(
    nwalkers, ndim, log_probability, args=(x, y, yerr)
)
sampler.run_mcmc(pos, 10000, progress=True);
```

```
(32, 4)
100%|██████████| 10000/10000 [00:07<00:00, 1270.09it/s]
```

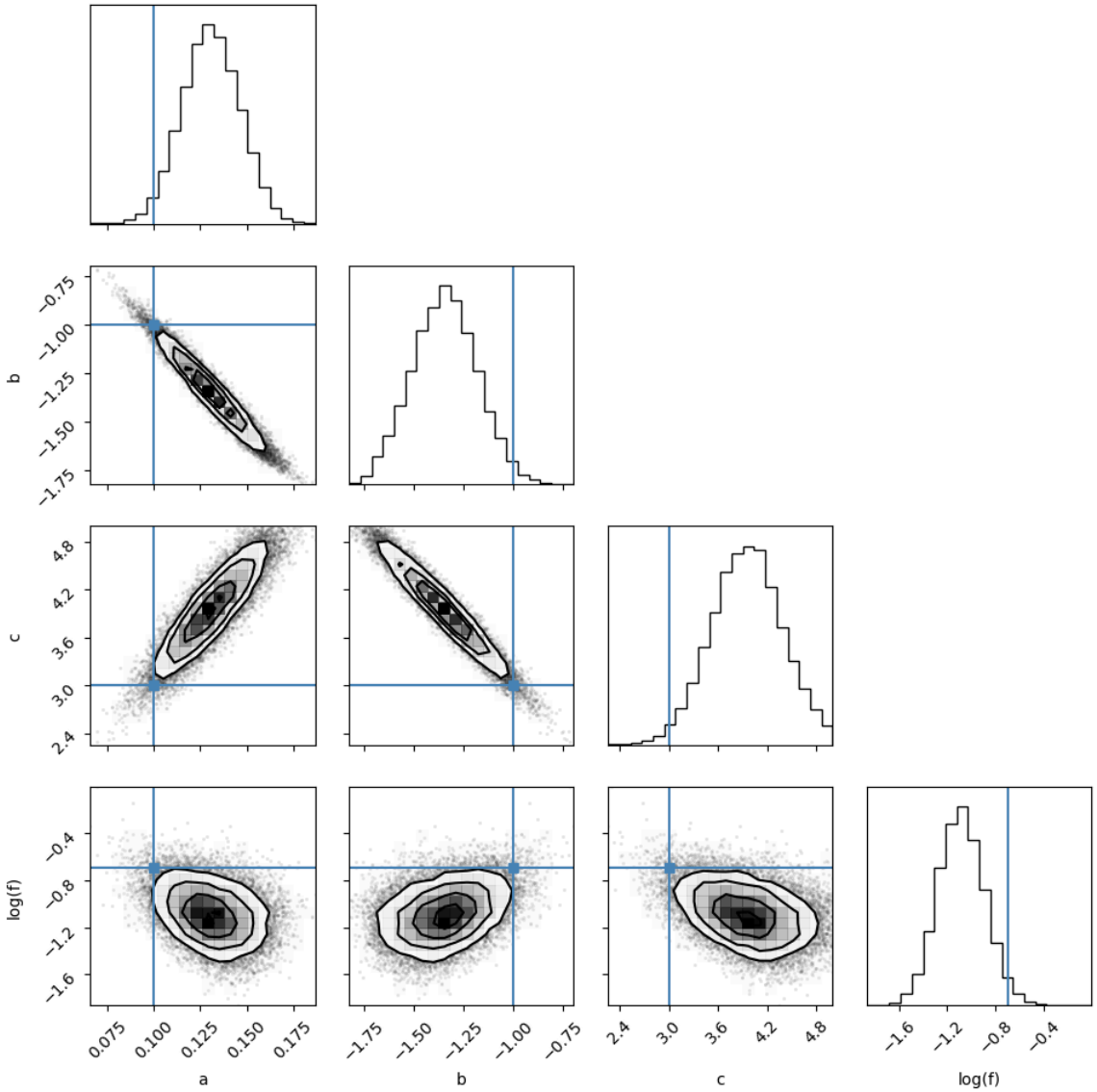
In [35]:
```
fig, axes = plt.subplots(4, figsize=(10, 7), sharex=True)
samples = sampler.get_chain()
labels = ["a", "b", "c", "log(f)"]
for i in range(ndim):
    ax = axes[i]
    # print(samples[:, :, i])
    ax.plot(samples[:, :, i], "k", alpha=0.3)
    ax.set_xlim(0, len(samples))
    ax.set_ylabel(labels[i])
    # ax.yaxis.set_label_coords(-0.1, 0.5)

axes[-1].set_xlabel("step number");

np.random.seed(132)

# Show the figure
plt.tight_layout()
```



In [36]:
```
flat_samples = sampler.get_chain(discard=100, thin=15, flat=True)
print(flat_samples.shape)
```

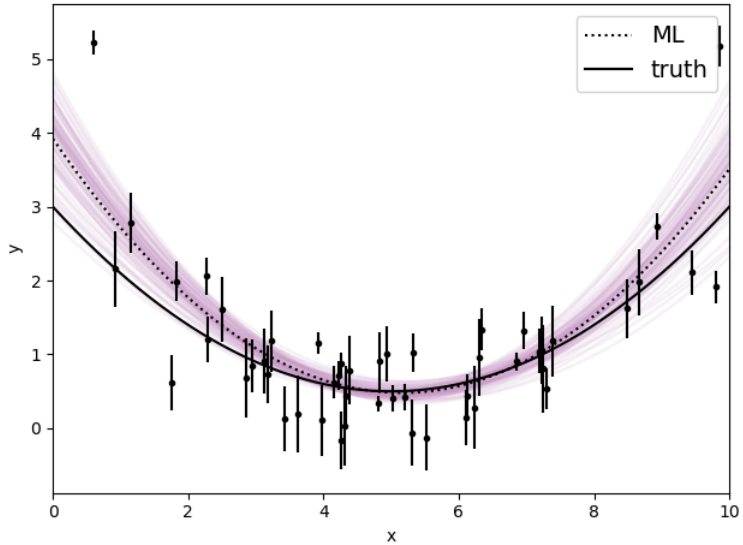
```
(21120, 4)
```

In [37]:
```
import corner

fig = corner.corner(
    flat_samples, labels=labels, truths=[a_true, b_true, c_true, np.log(f_true)]
);

# Show the figure
plt.tight_layout()
```



In [41]:

```
inds = np.random.randint(len(flat_samples), size=100)
for ind in inds:
    sample = flat_samples[ind]
    plt.plot(x0, np.dot(np.vander(x0, 3), sample[:3]), color = "#c994c7", alpha=0.1)
plt.errorbar(x, y, yerr=yerr, fmt=".k", capsize=0)
plt.plot(x0, np.dot(np.vander(x0, 3), [a_ml, b_ml, c_ml]), ":k", label="ML")
plt.plot(x0, a_true * x0**2 + b_true * x0 + c_true, "k", label="truth")
plt.legend(fontsize=14)
plt.xlim(0, 10)
plt.xlabel("x")
plt.ylabel("y");

# Show the figure
plt.tight_layout()
```



In [39]:
```
from IPython.display import display, Math

for i in range(ndim):
    mcmc = np.percentile(flat_samples[:, i], [16, 50, 84])
    q = np.diff(mcmc)
    txt = "\mathrm{{{3}}} = {0:.3f}_{{-{1:.3f}}}^{{{2:.3f}}}"
    txt = txt.format(mcmc[1], q[0], q[1], labels[i])
    display(Math(txt))
```

```
<>:6: SyntaxWarning: "\m" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\m"? A raw string is also an option.
<>:6: SyntaxWarning: "\m" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\m"? A raw string is also an option.
/tmp/ipykernel_5696/2909214736.py:6: SyntaxWarning: "\m" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\m"? A raw string is also an option.
  txt = "\mathrm{{{3}}} = {0:.3f}_{{-{1:.3f}}}^{{{2:.3f}}}"
```

$$\mathrm{a} = 0.130_{-0.015}^{0.016}$$

$$\mathrm{b} = -1.344_{-0.164}^{0.163}$$

$$\mathrm{c} = 3.949_{-0.434}^{0.422}$$

$$\mathrm{log(f)} = -1.086_{-0.186}^{0.193}$$